Theoretische Informatik 1
Woche 9

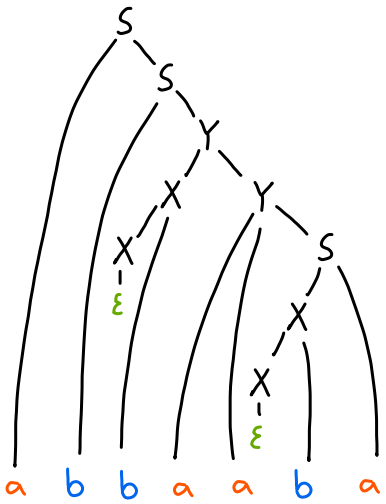
Lösungen zur Hausübung 2

▸ Aufgabe 1: CFG verstehen [3+3 Punkte]

Betrachten Sie die kontextfreie Grammatik (CFG) mit den Nichtterminalen N = {S, X, Y}, den Terminalen T = {a, b}, dem Startsymbol S und den folgenden Produktionsregeln:

S → aS | bY | Xa | X
X → ε | Xb
Y → XY | aaS

1. Zeigen Sie, dass der String w = "abbaaba" in der von dieser Grammatik erzeugten Sprache ist, indem Sie einen Ableitungsbaum für w zeichnen.



2. Geben Sie einen regulären Ausdruck an, der die Sprache beschreibt, die diese Grammatik erzeugt.

S → aS | bY | Xa | X
X → ε | Xb ⇦ b*
Y → XY | aaS

Informelle Zwischenschritte:

S → aS | bY | b*a | b*
Y → b*Y | aaS

S → aS | bY | b*a | b*
Y → b*aaS

S → aS | bb*aaS | b*a | b*

Achtung: das sind keine "richtigen" Grammatiken, da bei CFG auf der rechten Seite keine regulären Ausdrücke stehen dürfen.

Fertige Lösung:
L = (a|bb*aa)*b*(a|ε)

"außer am Ende darf nach einem b kein einzelnes a kommen"

Nicht in L ist z.B. bab

▸ **Aufgabe 2: CFG bauen** [3+3+3 Punkte]

Geben Sie für jede der folgenden Sprachen eine kontextfreie Grammatik (CFG) an, die genau die Strings dieser Sprache erzeugt. Dabei genügt es, wenn Sie die Produktionsregeln der Grammatik angeben, wobei S das Startsymbol sein soll.

1. Geben Sie eine CFG an, die folgende Sprache erzeugt:

$L = \{w \in \{a, b\}^* \mid$ w enthält *nicht* so viele a wie b, d.h. die Anzahl der Vorkommen von a in w ist ungleich der Anzahl der Vorkommen von b in w.$\}$

zunächst: Grammatik für die Sprache $L' = \{w \in \{a, b\}^* \mid$ w enthält genauso viele a wie b$\}$ (aus der letzten Vorlesung)

$S \rightarrow aSbS \mid bSaS \mid \varepsilon$

jetzt: Grammatik für die Sprache $L_{\leq} = \{w \in \{a, b\}^* \mid$ w enthält höchstens so viele a wie b$\}$

$S \rightarrow aSbS \mid bSaS \mid Sb \mid \varepsilon$

jetzt: Grammatik für die Sprache $L_{\geq} = \{w \in \{a, b\}^* \mid$ w enthält höchstens so viele a wie b$\}$

$S \rightarrow aSbS \mid bSaS \mid Sa \mid \varepsilon$

jetzt: Grammatik für die Sprache $L_{<} = \{w \in \{a, b\}^* \mid$ w enthält weniger a als b$\}$

$S \rightarrow XbX$ ⇦ extra b, damit Anzahl a < Anzahl b
$X \rightarrow aXbX \mid bXaX \mid Xa \mid \varepsilon$ ⇦ $L_{\leq}$

Lösung der Aufgabe: Grammatik für $L = \{w \in \{a, b\}^* \mid$ w enthält *nicht* so viele a wie b$\}$

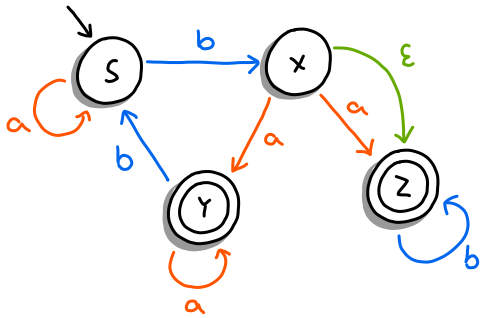
$S \rightarrow XbX \mid YaY$
$X \rightarrow aXbX \mid bXaX \mid Xb \mid \varepsilon$ ⇦ Anzahl a ≤ Anzahl b
$Y \rightarrow aYbY \mid bYaY \mid Ya \mid \varepsilon$ ⇦ Anzahl a ≥ Anzahl b

2. Geben Sie eine CFG an, die die Sprache erzeugt, die durch folgenden regulären Ausdruck beschrieben wird:

$(\varepsilon \mid a \mid b)\underbrace{(ab \mid ba)^*}_{X}$

$S \rightarrow X \mid aX \mid bX$
$X \rightarrow abX \mid baX \mid \varepsilon$

3. Geben Sie eine CFG an, die die Sprache erzeugt, die durch folgenden endlichen Automaten akzeptiert wird:



S → aS | bX
X → aY | aZ | Z
Y → aY | bS | ε
Z → bZ | ε

▸ **Aufgabe 3: Kellerautomat** [5 Punkte]

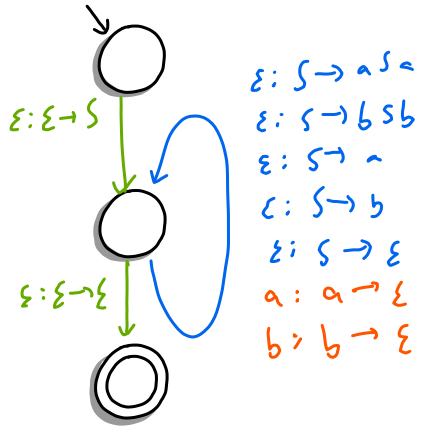
Konstruieren Sie einen Kellerautomaten, der die folgende Sprache akzeptiert:

L = {w ∈ {a, b}* | w ist ein Palindrom}

Ansatz 1:

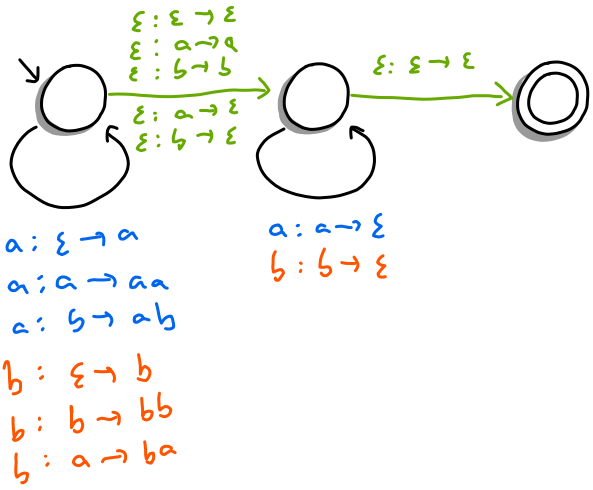
Zunächst CFG für L hinschreiben und dann nach "Schema F" in Kellerautomat umwandeln

S → aSa | bSb | a | b | ε



Ansatz 2:

Selbst Kellerautomat programmieren

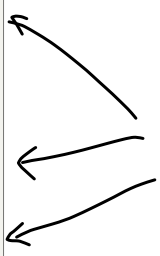


Loop

Programme bitte richtig einrücken!

```
loop x
   ⟨P⟩
end

if y
   ⟨P⟩
else
   ⟨P⟩
end
```



je um eine Position nach rechts einrücken!

Auch wichtig: Jedes *loop* hat ein *end*. Jedes *if* hat ein *end*.

Loop Extensions

weitere Sprachfeatures für extended Loop. Dürfen ab sofort verwendet werden.

▸ **Stringliterale**

"A"	code("A")
"Hamster"	code("Hamster")

Strings in Anführungzeichen stehen einfach als Abkürzungen für die Codes, in die sie kodiert sind.

▸ ***for*-Schleife**

Einfache *for*-Schleife mit fester Anzahl an Durchläufen (*loop*-like)
Achtung: *for* in Java oder C ist mächtiger! (entspricht *while*)

Seien x und y zwei Variablen oder Konstanten, x ≤ y

```
for i ← x, ..., y
   ⟨P⟩
end
```

```
i ← x
h ← y + 1
h ← h − x
loop h
   ⟨P⟩
   ++i
end
```

▸ **Anmerkung 1:** Es sollte x ≤ y sein!
▸ **Anmerkung 2:** in ⟨P⟩ sollte die Variable i nicht verändert werden!

bitte nicht so etwas schreiben:

```
x ← 0
for i ← 1, ..., 10
   x ← x + i
   −−i
end
```
⇦ **NEIN!!!**

▸ Komplexe Ausdrücke

Beispiel: x ← (3 · y · z + 4) · x ^ 2

Können z.B. infolgenden Situationen verwendet werden:

```
x ← ⟨Ausdruck⟩
loop ⟨Ausduck⟩
if ⟨Ausdruck⟩
return ⟨Ausdruck⟩
for i ← ⟨Ausdruck⟩, ..., ⟨Ausdruck⟩
```

Mögliche Ausdrücke:

⟨Ausdruck⟩ ::=
⟨Variable⟩ |
⟨Konstante⟩ |
⟨Stringliteral⟩ |
⟨op1⟩ ⟨Ausdruck⟩ |
⟨Ausdruck⟩ ⟨op2⟩ ⟨Ausdruck⟩ |
"(" ⟨Ausdruck⟩ ")"

⟨op1⟩ ::= "*not*"
⟨op2⟩ ::= "+" | "−" | "·" | "/" | "^" |"%" | "*and*" | "*or*" | "=" | "≠" | "<" | ">" | "≤" | "≥"

Anmerkung: Statt "·" dürfen Sie auch "*" schreiben. 🙂

Achtung: Keine ++ und −− in Ausdrücken!

Scheiben z.B. bitte statt

```
y ← 0
x ← ++y + 2
```


lieber:

```
y ← 0
++y
x ← y + 2
```


▸ Frage: ist

not x + y = *not* (x+y)

oder

not x + y = (*not* x) + y?

Antwort: Im Zweifelsfall lieber Klammer setzten!

▸ Beispielprogramm

```
counta ← 0
countb ← 0
for i ← 0, …, len(in)−1
    if in[i] = "a"
        ++counta
    end
    if in[i] = "b"
        ++countb
    end
end
return counta = countb
```

⇦ Hier ist der Notationszucker dieser Vorlesung in Aktion. 😎

▸ Was es in Loop nicht gibt…

(…, und was man darum auch nicht verwenden darf…)

1. andere Datentypen wie z.B. *float*
2. komplexe Datentypen wie Listen usw.
3. *break* oder *continue*
4. *while*
5. Rekursion

4. und 5. gehen über die Möglichkeiten von Loop hinaus.